

A Local-First Reading Room

Notes toward a shared-annotation device — architecture, hardware, and a build plan

A device that holds a small library and everyone's notes on it. People connect with the phones and laptops they already carry. Notes written while disconnected merge cleanly on reconnection, without a server deciding who wins. The library and the marginalia are files on a memory card, readable in fifty years without the software that made them.

Contents

- 1 The idea in plain language
 - 2 Why annotation is the easy case
 - 3 Data model and wire format
 - 4 Hardware: which ESP32, and why
 - 5 Networking, discovery, and the venue problem
 - 6 The node as a server: three endpoints
 - 7 Anchoring notes without manual work
 - 8 Local AI: what fits and what doesn't
 - 9 The spatial layer
 - 10 Physical media and sneakernet
 - 11 Access control by capability
 - 12 The cyberdeck enclosure
 - 13 An eight-week build plan
 - 14 A two-hour workshop
 - 15 Corpus, licensing, and framing
 - 16 Honest limits and open risks
 - 17 Prior art and reading
-

Written as a working reference. Numbers are approximate and worth re-verifying against current documentation before they become load-bearing.

1 The idea in plain language

Imagine a reading group where everyone owns the same book and writes in the margins. Normally those notes stay trapped in each private copy. Here, when people are in the same room, the notes appear in each other's copies. Nobody emailed anything. Nobody signed up for anything.

The little box

A cheap device about the size of a deck of cards holds the books and the notes. You connect to it the way you connect to café wi-fi; a page opens in your browser. You read, highlight, and write. Everyone else in the room is doing the same, and everyone sees everyone. No account, no password, no company in the middle.

Why nothing is ever lost

Say you read on a train with no signal and write eleven notes. A friend, also without signal, writes six. Later you are in the same room. The notes simply merge — all seventeen, nothing overwritten, no dialog asking you to choose a version.

The reason is almost too simple: **nobody ever changes anything**. You can only add. A note is a small permanent object — this person, this book, this spot, these words, this moment. Adding is safe. Two piles of notes combine the way two piles of leaves do. Even deletion is just adding a note that says *ignore that earlier one*. So there is never a fight about who wrote last, and nobody has to be in charge.

The book cannot wriggle

Every document gets a fingerprint calculated from its contents. Change one comma and it is a different fingerprint, a different document. A note therefore points not at “that book” loosely but at exactly those words in exactly that copy. Highlights on the open web drift because the page beneath them is edited; here the text is frozen and the note is nailed to it. And because the fingerprint comes from the file itself, every copy agrees without anyone assigning numbers or keeping a master list.

Sneakernet

If you are never in the same room as someone, mail them a memory card. Same notes, same merging. Slow, but it needs no infrastructure at all. This is also why the thing keeps working when the wi-fi dies, when the company folds, in fifty years: the books and the notes are just files. No app required to read them.

2 Why annotation is the easy case

Collaborative *text editing* is the hard version of this problem. Text CRDTs such as RGA carry roughly 20–60 bytes of metadata per character, so a 10 KB document becomes 200–600 KB of live state, plus tombstones that accumulate forever. That does not fit on a microcontroller and it is unpleasant anywhere.

Annotation dodges the whole problem. The document is **immutable** — a content-addressed blob that will never change. Annotations attach to it and never edit it. So an annotation is an immutable record with a globally unique identity of (`actor_id`, `lamport`). Adding is insertion; deleting is a tombstone record naming the target. That is an OR-Set with unique tags, it converges with about forty lines of code, and it requires no cleverness.

A second gift follows. Web annotation tools carry three redundant selectors — quote, prefix/suffix, position — because pages change underneath them. Your documents cannot change, so a byte range against a hash is sufficient and unambiguous forever. Re-anchoring, the hardest problem in that product category, is deleted by content addressing.

The clock problem

Microcontrollers have no battery-backed clock by default, and a monotonic timer resets on reboot. Any last-writer-wins scheme based on wall time is therefore broken. Use Lamport or hybrid logical clocks, and **persist the counter to non-volatile storage** — if you skip that, a reboot silently violates causality. A real-time clock module with a coin cell (or simply reading the clock from the first phone that connects) gives you human-readable dates; keep the logical counter for ordering regardless.

3 Data model and wire format

One record type for everything. A standalone note is an annotation whose target is the device's own notebook pseudo-document. Resist adding a second type — the branch propagates into every layer downstream.

<code>doc_hash[8]</code>	truncated hash of the target document
<code>start_u32</code>	byte offset into the document text
<code>len_u16</code>	byte length of the anchored span
<code>actor_u32</code>	device or browser identity
<code>lamport_u32</code>	logical clock, persisted across reboots
<code>kind_u8</code>	highlight note reply tombstone anchor
<code>parent[8]</code>	optional, for threading
<code>body[...]</code>	UTF-8

Roughly 27 bytes of header. A 150-character note fits comfortably inside a single small radio packet, which matters if a low-bandwidth transport is ever in play.

The log is the CRDT

Because records are immutable and uniquely identified, merging two sets is union. The storage layer therefore needs exactly two operations: *have I seen this id*, and *append*. Keep an in-memory hash set of seen ids (2,000 annotations is about 16 KB, rebuilt by scanning the log at boot) and append everything else to one length-prefixed file.

This yields a useful trick: **the sync cursor is a byte offset into that file**. A client says “I have up to byte 41,088” and the node seeks and streams the remainder. Serving a sync request becomes seek, read, send — no parsing, no merge function, almost no CPU. Note that this works only because a single node imposes one arrival order; node-to-node sync needs real set reconciliation.

4 Hardware: which ESP32, and why

The family matters more than people expect. The constraint that governs everything is memory bandwidth, not clock speed.

Chip	Radios	Memory	Why pick it
ESP32-S3	Wi-Fi 4, BLE 5	512 KB SRAM + up to 8 MB octal PSRAM	Native USB OTG, fast SDMMC, SIMD vector instructions. The default choice.
ESP32-C6	Wi-Fi 6, BLE 5, 802.15.4	512 KB, no PSRAM	Real Thread/IPv6 mesh routing. A relay, not a file server.
ESP32-C3	Wi-Fi 4, BLE 5	400 KB, no PSRAM	One to two dollars. Cheap fixed anchor nodes.
ESP32-P4	none (needs companion)	large, PSRAM	MIPI-DSI display work. Two chips; overkill here.
ESP32 (original)	Wi-Fi 4, BT Classic + BLE	520 KB + quad PSRAM	Only if you already own boards. No USB OTG.

Wi-Fi is 2.4 GHz only across most of the family. Practical throughput is 5–15 Mbps with protocol overhead; the network stack alone consumes 50–80 KB of RAM.

Why USB OTG decides it

Device mode lets the node present itself as **USB mass storage**. Plug it into any laptop and the library mounts as a drive — no app, no driver, no pairing. Host mode lets you plug in a keyboard, or a stick to ingest files from. For an object whose thesis is “carry your library,” that is the entire pitch in one port.

PSRAM matters for a second reason: a large e-ink framebuffer at 1200×825 in four-bit greyscale is roughly 495 KB, which does not fit in on-chip memory.

Hard limits worth memorising

- Sockets cap at 16 in the network stack; the bundled HTTP server defaults to 7 open.
- Flash is NOR: erase-before-write, 4 KB sectors, roughly 100k cycles. Batch writes; never commit per keystroke.
- Dual-slot over-the-air update halves your application partition.
- No memory management unit, no process isolation, no operating system. One heap; one bug takes down the device.
- External PSRAM runs at tens of MB/s and is cached — random access patterns thrash.

5 Networking, discovery, and the venue problem

Run access-point and station modes simultaneously, always

The node should permanently broadcast its own access point *and* attempt to join the building network. If the building network works, visitors keep their internet connection. If it fails, isolates you, or the venue says no, the fallback is already running and you lose no time. Both roles share one radio and therefore one channel; the access point inherits the station's channel.

Provision credentials through a configuration page on the node's own access point, saved to non-volatile storage, with several networks tried in turn. Hard-coded credentials mean a recompile at every new venue.

What breaks on someone else's network

- **Client isolation.** Most guest and enterprise networks block device-to-device traffic on the same SSID. If enabled, your node is invisible and there is no client-side workaround. Ask specifically whether it is on.
- **WPA2-Enterprise.** 802.1X with PEAP or EAP-TLS is supported in firmware but needs credentials, possibly a CA certificate, and permission for an unregistered device.
- **Click-through guest portals.** The node cannot agree to terms. Ask about MAC registration early; it can take days.
- **VLAN separation.** Staff and guest networks may not see each other even without isolation.
- **Multicast filtering.** Nice hostnames via mDNS often get dropped. Always display the raw IP address.

A pocket travel router in your bag is the best insurance available: a LAN you fully control, no isolation, no IT department, and a fixed network name you can print on a handout weeks ahead. Plan A is the building network, plan B is the travel router, plan C is the node's own access point. All three are the same firmware.

Captive portals are crippled browsers

When a phone joins an access point, the operating system probes for internet and, if intercepted, opens a *mini-browser* rather than the real one. That window has no persistent storage, dies when backgrounded, and cannot download files. Use it only to display the address and push people into their real browser, where offline queuing actually works.

Discovery without a directory

- **Connectionless radio beacons** for device-to-device: advertise identity and a state digest without joining anything.
- **Bluetooth advertising** for phones: a service identifier plus a short digest in the advertisement, visible without connecting.
- **The network name itself.** Broadcasting a recognisable SSID costs no code and lets a person standing in a room see the libraries around them in their wi-fi list.

6 The node as a server: three endpoints

Everything meaningful runs in the browser. The device stores and relays; it never parses an annotation body or resolves a conflict.

POST /ops	client sends ops; node appends unseen ones
GET /ops?since=N	node returns everything past byte offset N
GET /whoami	node assigns an actor id on first contact

Poll; do not use WebSockets

This is the single most consequential implementation decision for a room of twenty people. A WebSocket holds one socket per client for an entire session, and the socket limit is reached around the eighth participant. The failure presents as “the wi-fi is broken” in front of an audience.

Polling every two seconds holds a socket for milliseconds. Twenty people generate ten requests per second of mostly-empty responses, which is nothing. Two seconds of latency on an annotation appearing is imperceptible, and trading it away removes an entire class of session-ending failure.

Identity for browsers

Hand out a monotonic actor id from a persisted counter and let the person choose a display name. Store the id in a cookie, in local storage, and in the URL, because the captive mini-browser keeps none of them reliably.

The reassuring property: if a browser forgets its id and receives a new one, nothing breaks — that person simply appears as a new actor. **Identity churn costs user experience, never correctness.** This is worth demonstrating aloud; it is counterintuitive to anyone used to accounts.

Two things that bite

- **Bound the request body.** A client-controlled content length handed to an unbounded allocation is the classic way to crash a device with 512 KB of memory. Read in chunks into a fixed buffer; reject anything over a stated cap.
- **Let the phone be the clock.** A hidden form field carrying the browser's timestamp sets device time on first contact — free, and no clock module required.

Where offline actually happens

The clients are the offline participants, not the node. A phone queues annotations locally when it cannot reach the device and posts the batch on reconnection. That is the partition demonstration, and it is why the real browser tab matters more than the captive window.

7 Anchoring notes without manual work

Because the document is immutable and its full text is already on the card, anchoring is a **lookup problem, not an inference problem**. Run these in order and stop at the first confident hit; most notes resolve in the first two steps.

Method	Cost	Handles
Page number to byte range	Free, exact	“p. 47” — requires a page-offset table stored at ingest
Exact quoted fragment	Milliseconds	A phrase the reader copied down. Normalise whitespace, quotes, ligatures, and line-break hyphenation first
Rolling-hash shingles	~400 KB per book, milliseconds	Typos and transcription errors; five-word windows precomputed at ingest
Paragraph embeddings	Precomputed; a scan on device	Genuine paraphrase, where no literal overlap survives

Design the uncertainty into the interface

Return a ranked candidate list with scores, not a single answer. Auto-accept above a high threshold; below it, show two or three highlighted passages and let the reader tap one.

The elegant part, given the data model: **an unanchored note is still a valid annotation**. Target the document as a whole, mark it pending, and let it sync normally. Anchoring becomes a separate, later operation that any device can perform and gossip back. A note captured on a device with no spare capacity can be anchored by a laptop an hour later. The small device never has to be good at this; it only has to not lose the note.

Handwriting

Handwriting recognition from images does not fit on a microcontroller — modern models run to hundreds of millions of parameters, and it is the dense compute rather than the lookup tables that fails to fit. What does fit: page detection and focus gating during capture, deskew and binarisation, and isolated digit or checkbox recognition.

One important exception: **stylus input is a far easier problem than photographed handwriting**, because capturing the stroke sequence removes most of the ambiguity. A small recurrent model over pen coordinates runs comfortably on-device. If handwriting genuinely matters, a digitiser beats a camera by an enormous margin. Otherwise, capture the image as a blob and transcribe at ingest on a real machine, storing the text as a derived object keyed by the image's hash.

8 Local AI: what fits and what doesn't

Language models on this class of chip are now a genuine engineering achievement and still a toy capability. Models around 29 million parameters run at roughly 9–10 tokens per second by keeping most parameters in a flash lookup table that is read from rather than multiplied against. They write short simple stories. They cannot answer questions, follow instructions, or recall facts, and that ceiling is the dense reasoning core rather than a tuning problem.

The lesson underneath

One result makes it vivid: shrinking a model's vocabulary made it *faster*, because the output layer's weights are staged from slow external memory on every single token. **The currency on this platform is bytes moved per operation, not arithmetic**. Anything precomputed once and reduced to a small read at query time will fly; anything streaming megabytes per interaction will crawl regardless of how clever the mathematics is.

What actually runs well

Task	Verdict	Notes
Wake word, small command set	Production-grade	Ships in commercial products today
Audio event classification	Solid	Small network over spectrograms
Face or person detection	Solid	96×96 to 240×240, a few frames per second
Vector similarity search	Excellent	The one that matters here — see below
Optical character recognition	Poor	Do it at ingest; store text as a derived blob
Instruction-following model	No	Not a tuning problem

The idea that actually fits: precompute, then search

Embedding a document is expensive and happens once. Searching is cheap and happens constantly. Split them across machines and the arithmetic turns in your favour: five thousand documents at 384 dimensions in eight-bit integers is 1.9 MB, which sits in external memory comfortably, and an exhaustive scan takes tens of milliseconds using the chip's vector instructions. Binary embeddings with a bit-count distance shrink that to 240 KB — small enough for a chip with no external memory at all.

What makes this elegant rather than merely clever: **an embedding is a property of an immutable content-addressed blob**. Compute it once, anywhere, ever. Store it keyed by the document's hash. Peers hand each other embeddings alongside documents and nobody recomputes anyone else's work.

Distributed inference is a trap; distributed search is not

Four small chips with two megabytes each give you the same weight capacity as one chip with eight, but now with network hops between layers. Several devices do not make a bigger model; they make several copies of a small one.

Three distributed things that do work:

- **Sharded vector search.** Search is embarrassingly parallel and results are tiny. Each node searches its own library; a query fans out and each returns a few dozen bytes of matches. Walk into a room, ask what anyone here has on a subject, get answers from five libraries in under a second, with nothing leaving anyone's device but scores.
- **Inference at the edge, conclusions on the wire.** Many cheap nodes each running a tiny classifier and gossiping a label rather than raw data.
- **Gossip learning.** Nodes train locally and exchange weight deltas; averaging is commutative, which rhymes with everything else in this design.

Graceful degradation

Semantic search works always, on every device, at zero marginal cost. Summarisation and question answering work *when a real machine is in range* — a laptop, a small server, someone's phone — over the same discovery mechanism already being built. That is a feature rather than a gap: the device is complete on its own and grows more capable in company, which is also what the project is about.

9 The spatial layer

The optional but compelling front end: a pre-rendered background image, sprite characters walking on an invisible collision mask, and data placing doors, documents, and items. This is precisely how the pre-rendered console role-playing games of the late nineties worked — the aesthetic you want is the cheap technique, not the expensive one. You pay the three-dimensional cost once, offline, and ship an image.

What a room is, concretely

- **A background image.** 1024×768 in a modern format is 80–200 KB.
- **A walk mask.** One-bit image or polygon list — painted in an image editor.
- **Depth or sort order** so a character passes behind furniture.
- **A scale rule** so characters shrink toward the back; two values and interpolation covers most rooms.
- **Hotspots** — named polygons.
- **Portals** — polygon plus target room.

Everything but the images is a two to five kilobyte text file. Write the format down before writing code; it is the contract between renderers.

The idea that makes it a file system rather than a level

A hotspot does not *contain* items. It **binds to a path prefix**, and whatever lives at that prefix renders as items. A room becomes a view over a namespace, not a container of objects.

Two consequences. First, the same background art can be mounted at different paths by different people — one artist draws a good library, a hundred people live in it over their own files. That is the only way art stays affordable to produce. Second, the file system is the source of truth and the room can never desynchronise from it: drop a document into the folder from a terminal and it appears on the shelf. That is the demonstration.

Appearance for free

Choose the item sprite by file type — a scroll, a framed print, a bound volume. Choose the *variant* deterministically from the content hash, so the same file looks identical on every peer forever with no stored metadata and no coordination. Content-addressed appearance falls straight out of content-addressed storage.

The one that matters most: a blob you can see but cannot decrypt should still render, as a sealed object — present but opaque. That makes the permission model *visible*, which teaches far better than a dialog box.

Movement, and why it is cheap

Click-to-move destroys the traffic problem. Send a destination rather than a position: one message per click, every client runs the same path-finding over the shared mask and interpolates. Continuous position broadcast at ten updates per second scales as the square of the player count and will not survive a small device; destinations are a message every few seconds per person.

Note also that presence is **not** annotation. Live positions are ephemeral, belong on an unreliable channel, and must never touch storage. Persistent world state is the CRDT; cursors and avatars are not.

Layering: different clients, not quality settings

The world layout must be data, never code. If any of it lives in the renderer, the tiers diverge and you maintain three worlds. Done correctly, a text client is not a degraded mode but a legitimate reading of the same data — which also yields an accessibility story and a scripting story for free.

10 Physical media and sneakernet

Treating physical delivery as a first-class transport forces one specific decision: the sync protocol must be **set reconciliation** rather than a session. Exchange a fingerprint over a range of the namespace, bisect recursively where fingerprints disagree. That works identically over a socket, a memory card, or a printed code. Anything connection-oriented dies the moment the connection is a person walking across a city.

Write-once media maps unusually well onto immutable content-addressed blobs. A disc cannot be corrupted by a partial write, cannot wear out, and cannot drift from its hash. It is a frozen, verifiable shard of the store — a physical object whose contents are provably what its manifest claims. That is a stronger archival guarantee than the drive in your laptop, and it connects directly to catalogued-object thinking in archival practice.

The archival discipline

The on-disk format must be readable without your software. A directory of blobs named by hash, a manifest, a signature. If someone in 2050 needs your program to read the disc, the design has failed the test that motivated it. Rewritable media carries the mutable log; write-once media carries frozen content.

11 Access control by capability

An access-control list requires a trusted server that can refuse. There is not one. If someone holds the bytes, no permission bit stops them reading. Any design where “access rights” means a field the client is expected to check is theatre — the first non-conforming client owns everything.

The model that survives contact

- **Read access is holding a decryption key.** Encrypt the blob; address it by the hash of the ciphertext. Sharing means handing over a hash and a key. Someone without the key can store and relay the file forever and learn nothing.
- **Write access is signature verification.** You cannot stop a fork. You can decide whose signed writes you merge. That is trust, not access control — a different verb, and the honest one.
- **Delegation is a signed capability chain.** A holder signs a narrower, expiring capability for someone else; anyone can verify the chain without contacting the grantor.

What genuinely does not work

Revocation. You can grant; you cannot un-grant. Anything read is read forever. The realistic mitigations are expiring capabilities, key rotation under a new epoch, and accepting that revocation governs future content rather than past. Say this plainly in the design notes; it is a feature of honesty.

The shortcut for version one

A shared-key radio channel is a capability, and a serviceable one: a reading group is a channel, and sharing the channel grants access to that group's annotations. It is cryptographically real, costs nothing to build, and has honest limits — no per-document granularity, no revocation without re-keying everyone. Ship that, name the limits, and build the capability chain later.

12 The cyberdeck enclosure

E-ink and animation are incompatible

Not slow — incompatible. A full refresh is one and a half to four seconds with a visible inversion flash; partial refresh reaches three to five hundred milliseconds with accumulating ghosting; the fastest monochrome mode leaves residue requiring periodic full clears. There is no configuration in which a character walks smoothly across a room.

The resolution is the tiering paying off: **the e-ink deck is the text-tier client**. It reads, annotates, lists shelves, and describes rooms in words. The spatial view lives on a backlit screen or in a visitor's browser. Same protocol, same store, different renderer. One deck doing both means giving up multi-week battery life; choose which property you are actually buying.

What goes in the box

Subsystem	Bus	Note
E-ink panel	SPI or parallel	Small panels are simple; large salvaged panels need external memory for the framebuffer and give greyscale
microSD	SDMMC 4-bit	Four-bit rather than serial matters once you serve real files
Keyboard	I2C or USB host	Handheld keyboard modules, or any USB keyboard
Real-time clock	I2C	Two dollars; solves human-readable timestamps
USB-C	OTG host + device	Mass-storage mode is the feature. Also ingest from a stick
Audio	I2S DAC	Compressed audio decodes fine on a spare core for a standalone deck
Camera	parallel	Page capture makes the deck a producer, not only a consumer
Battery management	I2C	A real charge controller and fuel gauge, not a bare module

Build order

Buy an existing handheld first. A pre-made keyboard device gets firmware running on real hardware while the design is still moving; the software is the long pole. **Then breadboard** the e-ink version on development boards. **Then print an enclosure** around those boards. Spin a custom circuit board only once the layout has stopped changing, which is later than it feels.

On power: months in deep sleep with e-ink holding its last page at zero draw, roughly 15–25 hours of reading, and four to eight hours with the radio hot. That gap is the entire argument for the platform, and it justifies a physical radio switch — both for battery life and because a hardware toggle meaning *this object is not talking to anything* is a real gesture.

13 An eight-week build plan

The riskiest item is not the merge logic. It is whether the radio stack, a web server, and card storage coexist on one board without a month spent inside someone else's codebase. **Prove that in week one, before designing anything.**

Week	Ship this	Demonstrable as
1	Spike only. Confirm networking, card read/write, and file serving coexist in one build.	A log line proving a custom message arrived
2	Record format and append-only log on the card. Logical clock persisted. No networking.	A note created over serial survives reboot
3	Broadcast on create, append on receive, deduplicate by identity.	Annotate on one device, watch it appear on another
4	Catch-up: advertise a digest, request missing ranges.	A device that was off comes back and fills in
5	The reader. Portal serves document plus annotation overlay; select text to annotate.	Browser highlight propagates to another browser
6	Freeze. Bug fixes, error paths, README, demo video. Nothing new.	A finished thing with a URL
7	One room: background, mask, sprite, click-to-move, one shelf bound to a folder.	Walk to the shelf, see your documents
8	Music: a hotspot lists the folder, the browser plays the file. Polish.	Walk in, put a record on, read a book

Week six is the load-bearing row. The annotation device is a complete, novel, demonstrable artefact on its own. If the final fortnight slips, you still shipped. Without the freeze, this becomes an eighteen-month project.

Cut list, stated explicitly

No bulk transfer over a low-bandwidth radio. No text editing, ever — annotations only. No three-dimensional rendering. No capability system beyond shared channel keys. No world beyond one room. **No level editor** — an image editor and a text editor are the editor for version one, and possibly for version three. No item art beyond four file-type icons.

On audio, a correction worth carrying

When serving to a browser, do not decode audio on the device — serve the file with range-request support and let the browser play it. That is a thirty-line feature instead of a multi-week detour. For a *standalone* deck with its own audio hardware, on-device decoding is entirely reasonable on a spare core. The two cases have opposite answers.

14 A two-hour workshop

Do not hand out devices

Own four to six nodes as a permanent loaner kit; participants bring phones and laptops. Flashing firmware across fifteen unfamiliar machines reliably consumes forty-five minutes and strands half the room on driver problems. Everything ships pre-flashed and nobody compiles anything.

It is also thematically correct: the nodes are the furniture of the room, not party favours. And a bare development board is a poor souvenir — no screen, no enclosure, no purpose, straight into a drawer.

Give people a memory card instead. A few dollars each, loaded with the firmware image, the source, the room definition, and a starter library. It is the thesis in object form: you leave with data on physical media, not a link to a server.

Two hours, honestly

Time	What happens
0:00–0:15	Join, open the page, see the library. Payoff in the first ten minutes.
0:15–0:35	Everyone annotates the same text and watches each other's marks appear.
0:35–0:55	Partition and heal. Take clients offline, keep writing on both sides, reconnect.
0:55–1:10	Break.
1:10–1:35	One contained modification, edited in a browser form — not a toolchain.
1:35–1:55	Sneakernet: walk a card between two nodes that have never met. Hand out cards.
1:55–2:00	Repository, specification, where this goes.

The partition demonstration is the entire workshop; everything else is setup for it. If you run late, cut the modification exercise, cut the room layer, cut the talk — protect those twenty minutes.

Staging the partition on a shared network

You cannot power down the venue's access point. Make the disconnection a *device-side* action instead — a button or physical switch that stops the radio. That is more legible anyway: the audience sees a node deliberately taken off the network rather than someone unplugging something in a closet.

15 Corpus, licensing, and framing

Curate the library; do not let participants upload. This resolves the rights question and a large time sink at once, and it is defensible on its own terms: everyone reading the same texts is what makes annotations comparable and the conversation coherent.

Unambiguous sources

- **Standard Ebooks** — public domain, professionally typeset. The best single source for a workshop.
- **PubMed Central Open Access subset** — explicitly marked for redistribution.
- **PLOS** — Creative Commons attribution throughout.
- **OpenStax** — openly licensed textbooks.
- **Wikisource, Project Gutenberg, and public-domain library collections.**
- **United States federal documents** — public domain by statute.
- **Preprint servers need care** — many papers carry a non-exclusive distribution licence rather than an open one. Check per item; open-access directories are safer in bulk.

Make licensing a feature

Since blobs are content-addressed and immutable, **attach a licence identifier to each one as first-class metadata** and render it in the interface — a small tag on the spine. Provenance becomes something you demonstrate rather than something you apologise for. A sharing system that carries its own rights information is a better technical claim than a sharing system, it is a concern archivists recognise immediately as serious, and it converts the awkward part of the talk into the interesting part.

The complementary framing: **the texts are the corpus; the annotations are the work**. What participants create and share is their own commentary — their expression, their copyright, and the genuinely novel part of the system. Short quotations inside annotations are ordinary quotation. That line is clean and sharper intellectually than the alternative.

Positioning

Be deliberate about the name. *Local-first collaborative annotation* or *reading together offline* places the work in the lineage of marginalia, commonplace books, and scholarly apparatus. *Mesh file sharing* places it somewhere else entirely and shapes both what people bring on the day and what the venue worries about.

Two practical additions: wipe the nodes after each session and say so at the start, which removes the rights residue and the privacy question in one sentence. And if the venue is a library or a university, co-facilitate with a librarian — they know licensing better than any engineer, it substantially de-risks the event, and it is a good relationship to have.

16 Honest limits and open risks

A single node is a server

With one central device you do not strictly need a mergeable data type; a server-assigned sequence number would work for anyone who stays connected, and would be simpler. The device is a single point of failure and everything routes through it.

What the design buys is that the *protocol* does not require one — demonstrated by taking clients offline, and later by adding a second node or walking a card across town. Say that plainly rather than letting “local-first” imply “no server.” The stronger claim is the true one: the server is an optimisation here, not a dependency.

Visiting versus owning

A visitor's browser holds a real replica — that is how offline writing works. What it lacks is persistence the visitor controls: clearing site data wipes it, a different browser is a different copy, and a phone cannot host others.

The rung between them is **the export**. A visitor downloads their annotations as a file, or writes them to a card, and that file is a first-class replica that merges into any node later. So the ladder runs: browser tab, exported file, memory card, owned device. **The protocol should not care which rung you are on**. Built that way, universal ownership stops being a threshold and becomes a preference.

Which reframes the product question. Not *how do we get devices to everyone*, but *what does owning one give you that visiting does not*: your library is yours and travels with you; you can host rather than always be a guest; it works with no phone, no signal, and no charge for weeks; and it holds things you would never upload anywhere. None of those is synchronisation, which visitors get free.

Where a bigger machine wins

A small single-board computer is roughly a hundred times the machine for similar money, runs a real operating system, and would let you deploy mature libraries in a weekend. The arguments that survive are specific and worth holding consciously: instant boot with nothing to corrupt on a yanked cable, milliamps rather than watts, mass-storage mode, and an object that can be left in a building for a year with nothing to patch.

A defensible sequence: prototype where iteration is fast, freeze the protocol, then port down. The constrained implementation is far more impressive as *and it also runs on an eight-dollar microcontroller* than as the thing that took eight weeks to get one room working.

The framing that makes it coherent

A microcontroller deck is not a small computer. There is no shell and no arbitrary program execution; every capability is compiled in ahead of time. What it can be is an **instrument** — a single-purpose object that does one thing precisely and runs for a month on a battery. A device whose entire job is holding a library, annotating it, and finding other libraries in the room is exactly that, and a far more interesting object than an underpowered laptop.

17 Prior art and reading

Area	Look at
Sync and capabilities	The Willow protocol and its Meadowcap capability system — path-structured namespaces, partial sync, transport-agnostic, range-based set reconciliation. Closest existing thing to this design.
Encrypted content addressing	Tahoe-LAFS read-capability and write-capability model; capability-chain specifications for delegation.
Practical implementations	Blob-and-document sync libraries in Rust; browser-side mergeable data types for the client half.
Delay-tolerant messaging	Long-range mesh messaging projects — proven store-and-forward on this chip family, worth reading before designing a protocol.
Spatial multi-user systems	Object-oriented multi-user dungeons of the early nineties for ownership and permission semantics; browser-based virtual office tools for the modern presentation.
Pre-rendered backgrounds	Late-nineties console role-playing games, point-and-click adventure engines, and isometric social worlds — the exact technique, well documented.
Annotation as a practice	Web annotation data models, social reading platforms, and the older tradition of marginalia and commonplace books.

Figures throughout are order-of-magnitude and drawn from a design conversation rather than from measurement. Verify anything before it becomes structural — particularly memory budgets, radio behaviour, and platform-specific limits, which change between chip revisions and firmware releases.